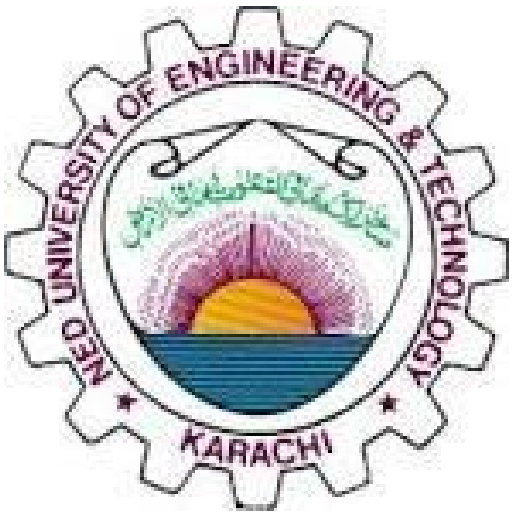
OBJECT ORIENTED PROGRAMMING [CT-260]



NAME: ABDUL RAHEEM SHEIKH

ROLL NO: CT-25192

DEPARTMENT: BCIT

BATCH: 2025

YEAR & SECTION: FSCS-D

LAB: 7

CODES
QUESTION 01

```
#include <iostream>
using namespace std;

class Matrix {
private:
    int rows, cols;
    int **arr;

public:
    // Default Constructor
    Matrix() {
        rows = cols = 0;
        arr = nullptr;
    }

    // Parameterized Constructor
    Matrix(int r, int c) {
        rows = r;
        cols = c;
        arr = new int*[rows];
        for(int i=0; i<rows; i++)
            arr[i] = new int[cols]{0};
    }

    // Copy Constructor
    Matrix(const Matrix &obj) {
        rows = obj.rows;
        cols = obj.cols;

        arr = new int*[rows];
        for(int i=0; i<rows; i++) {
```

```
            arr[i] = new int[cols];
            for(int j=0; j<cols; j++)
                arr[i][j] = obj.arr[i][j];
        }
    }

    // Assignment Operator
    Matrix& operator=(const Matrix &obj) {
        if(this != &obj) {
            for(int i=0; i<rows; i++)
                delete[] arr[i];
            delete[] arr;

            rows = obj.rows;
            cols = obj.cols;

            arr = new int*[rows];
            for(int i=0; i<rows; i++) {
                arr[i] = new int[cols];
                for(int j=0; j<cols; j++)
                    arr[i][j] = obj.arr[i][j];
            }
        }
        return *this;
    }

    // + Operator
    Matrix operator+(const Matrix &obj) {
        Matrix temp(rows, cols);
        for(int i=0; i<rows; i++)
            for(int j=0; j<cols; j++)
                temp.arr[i][j] = arr[i][j] + obj.arr[i][j];
        return temp;
    }

    // - Operator
    Matrix operator-(const Matrix &obj) {
```

```
    Matrix temp(rows, cols);
    for(int i=0; i<rows; i++)
        for(int j=0; j<cols; j++)
            temp.arr[i][j] = arr[i][j] - obj.arr[i][j];
    return temp;
}

// * Operator
Matrix operator*(const Matrix &obj) {
    Matrix temp(rows, obj.cols);

    for(int i=0; i<rows; i++) {
        for(int j=0; j<obj.cols; j++) {
            temp.arr[i][j] = 0;
            for(int k=0; k<cols; k++)
                temp.arr[i][j] += arr[i][k] * obj.arr[k][j];
        }
    }
    return temp;
}

// Indexing Operator
int* operator[](int index) {
    return arr[index];
}

// Destructor
~Matrix() {
    for(int i=0; i<rows; i++)
        delete[] arr[i];
    delete[] arr;
}

void input() {
    for(int i=0; i<rows; i++)
        for(int j=0; j<cols; j++)
            cin >> arr[i][j];
```

```
    }

    void display() {
        for(int i=0; i<rows; i++) {
            for(int j=0; j<cols; j++)
                cout << arr[i][j] << " ";
            cout << endl;
        }
    }

    int getRows() { return rows; }
    int getCols() { return cols; }
};

// Test Program
int main() {
    Matrix A(2,2), B(2,2);

    cout << "Enter Matrix A:\n";
    A.input();

    cout << "Enter Matrix B:\n";
    B.input();

    Matrix C = A + B;
    cout << "\nAddition:\n";
    C.display();

    Matrix D = A - B;
    cout << "\nSubtraction:\n";
    D.display();

    Matrix E = A * B;
    cout << "\nMultiplication:\n";
    E.display();

    cout << "\nAccess A[0][1] = " << A[0][1];
```

```
    return 0;
}
```

QUESTION . 02

```
#include <iostream>
using namespace std;

bool searchMatrix(int matrix[][4], int rows, int cols, int target) {
    int low = 0, high = rows * cols - 1;

    while(low <= high) {
        int mid = (low + high) / 2;

        int r = mid / cols;
        int c = mid % cols;

        if(matrix[r][c] == target)
            return true;
        else if(matrix[r][c] < target)
            low = mid + 1;
        else
            high = mid - 1;
    }

    return false;
}

int main() {
    int matrix[3][4] = {
        {1,3,5,7},
        {10,11,16,20},
        {23,30,34,60}
    };

    int target;
    cout << "Enter target: ";
    cin >> target;
```

```
    if(searchMatrix(matrix,3,4,target))
        cout << "True";
    else
        cout << "False";

    return 0;
}
```

QUESTION . 03

```
#include <iostream>
using namespace std;

class Payroll;

class Employee {
private:
    string name;
    string designation;
    double salary;

public:
    Employee(string n, string d, double s) {
        name = n;
        designation = d;
        salary = s;
    }

    friend class Payroll;

    void show() {
```

```
        cout << "Name: " << name << endl;
        cout << "Designation: " << designation << endl;
        cout << "Salary: " << salary << endl;
    }
};

class Payroll {
public:
    void updateSalary(Employee &e, double amount) {
        e.salary += amount;
    }
};

int main() {
    Employee emp("Ali", "Manager", 50000);
    Payroll p;

    cout << "Before Update:\n";
    emp.show();

    p.updateSalary(emp, 10000);

    cout << "\nAfter Update:\n";
    emp.show();

    return 0;
}
```

QUESTION . 04

```
#include <iostream>
using namespace std;

class Employee;

void updateSalary(Employee &, double);

class Employee {
private:
    string name;
    string designation;
    double salary;

public:
    Employee(string n, string d, double s) {
        name = n;
        designation = d;
        salary = s;
    }

    friend void updateSalary(Employee &, double);

    void show() {
        cout << "Name: " << name << endl;
        cout << "Designation: " << designation << endl;
        cout << "Salary: " << salary << endl;
    }
};

void updateSalary(Employee &e, double amount) {
    e.salary += amount;
}
```

```
int main() {
    Employee emp("Sara", "HR", 40000);

    cout << "Before Update:\n";
    emp.show();

    updateSalary(emp, 5000);

    cout << "\nAfter Update:\n";
    emp.show();

    return 0;
}
```

OUTPUTS

QUESTION .01

Output Clear

```
Enter Matrix A:
1 2
5 6
Enter Matrix B:
7 8
6 9

Addition:
8 10
11 15

Subtraction:
-6 -6
-1 -3

Multiplication:
19 26
71 94

Access A[0][1] = 2

=== Code Execution Successful ===
```

QUESTION . 02

Output

```
Enter target: 3
True

=== Code Execution Successful ===
```

```
Enter target: 13
False

=== Code Execution Successful ===
```

QUESTION . 03

Output

```
Before Update:
Name: Ali
Designation: Manager
Salary: 50000

After Update:
Name: Ali
Designation: Manager
Salary: 60000


=== Code Execution Successful ===
```

QUESTION . 04

Output

```
Before Update:
Name: Sara
Designation: HR
Salary: 40000

After Update:
Name: Sara
Designation: HR
Salary: 45000


=== Code Execution Successful ===
```